Computer Organisation

Topic 3

Computer Arithmetic

Stallings, Computer Organization & Architecture (9th Ed.), Chap. 10

Outline

1. Arithmetic & Logic Unit

2. Integer Representation
   - Sign-magnitude
   - Twos complement
   - Conversion between lengths

3. Integer Arithmetic
   - Addition & Subtraction
   - Multiplication
   - Division

4. Floating-point Arithmetic

Learning objectives

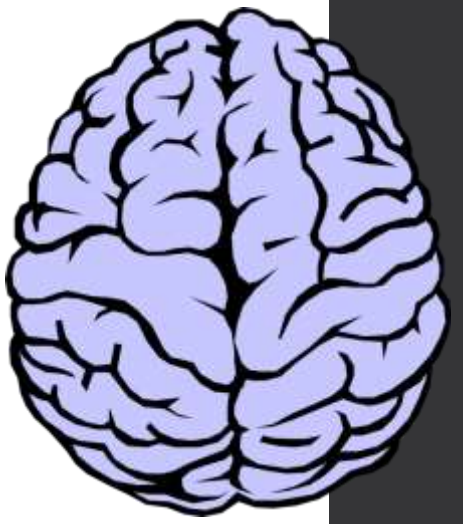
Understand the distinction between the way in which numbers are represented (the binary format) and the algorithms used for the basic arithmetic operations.

Explain twos complement representation.

Present an overview of the techniques for doing basic arithmetic operations in twos complement notation.

Understand the use of significand, base, and exponent in the representation of floating-point numbers.

Arithmetic & Logic Unit



- A vital component of the CPU responsible for executing arithmetic and logical instructions
- A fundamental part of a computer's processing capabilities
- Other elements of the computer system (CU, registers, memory) mainly to bring data into the ALU for it to process and then to take the results back out
- Does the **calculations**
- Everything else in the computer is there to service this unit

Image source: https://www.wannapik.com/vectors/10124

Arithmetic & Logic Unit

- Handles **integers**
- May handle **floating point** (real) numbers

ALU Inputs and Outputs

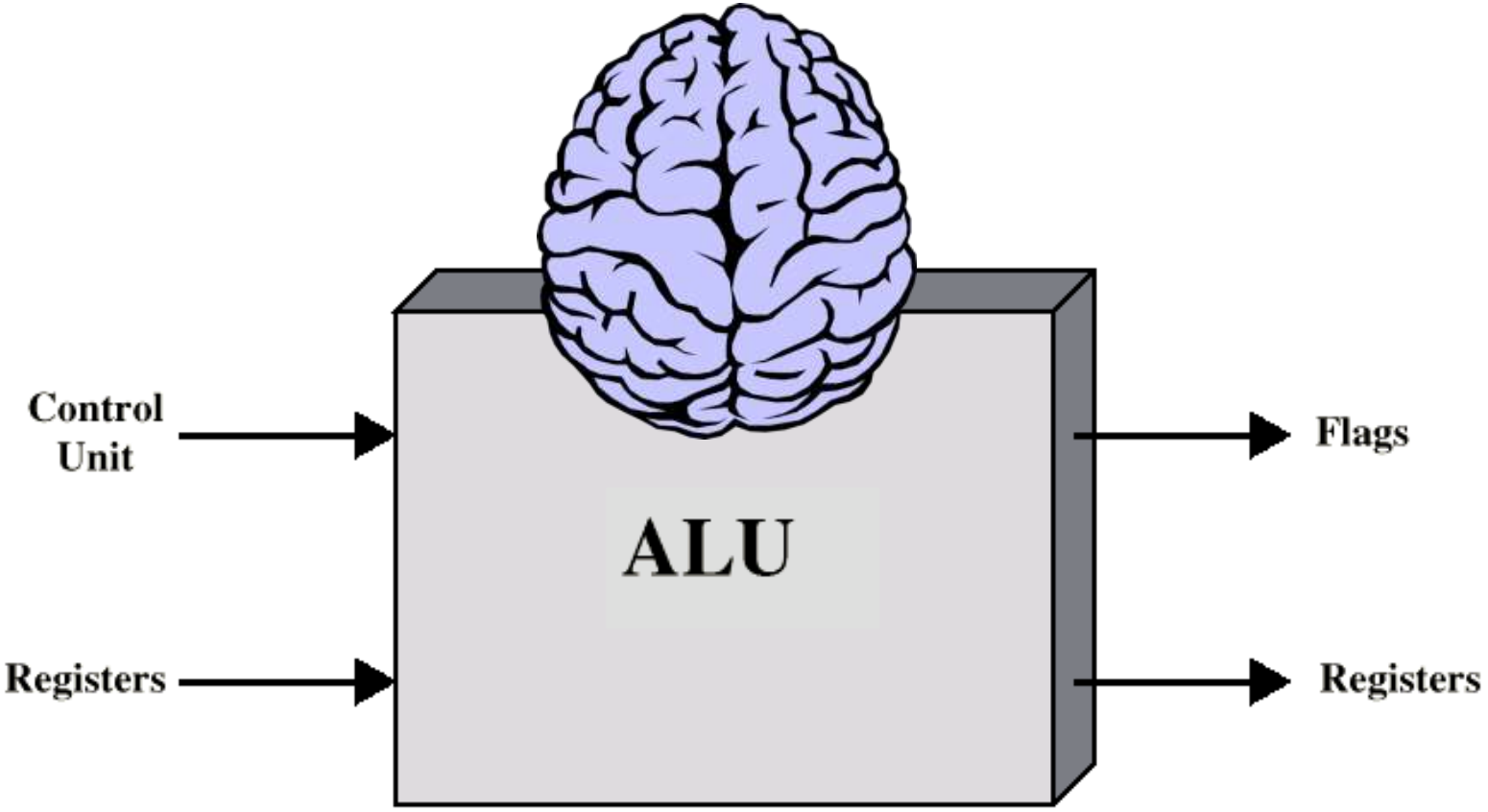


Image source: https://www.wannapik.com/vectors/10124

Outline

1. Arithmetic & Logic Unit

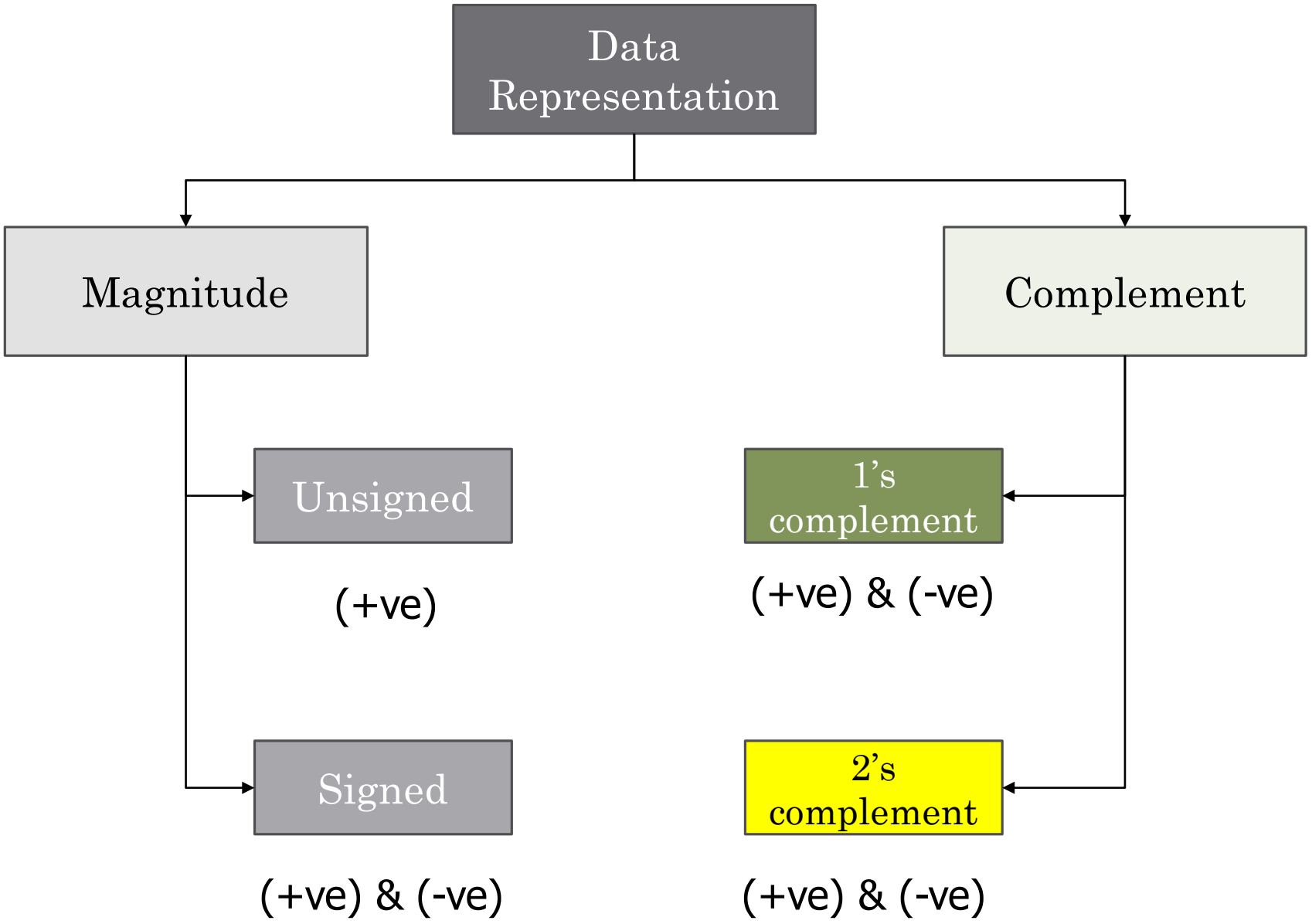
2. Integer Representation
   - Sign-magnitude
   - Twos complement
   - Conversion between lengths

3. Integer Arithmetic
   - Addition & Subtraction
   - Multiplication
   - Division

4. Floating-point Arithmetic

Integer Representation

- The way in which whole numbers (integers) are represented in a digital computer/binary system
- Only have **0** & **1** to represent everything
- Positive numbers stored in binary
  - e.g. 41=00101001
- Binary digit or a **Bit**



In all representation, (+) number is represented in the same way

Integer Representation

- In the binary system, number system can be represented with just the digits 0 and 1

An 8-bit word can represent the numbers from 0 to 255, such as

00000000 = 0

00000001 = 1

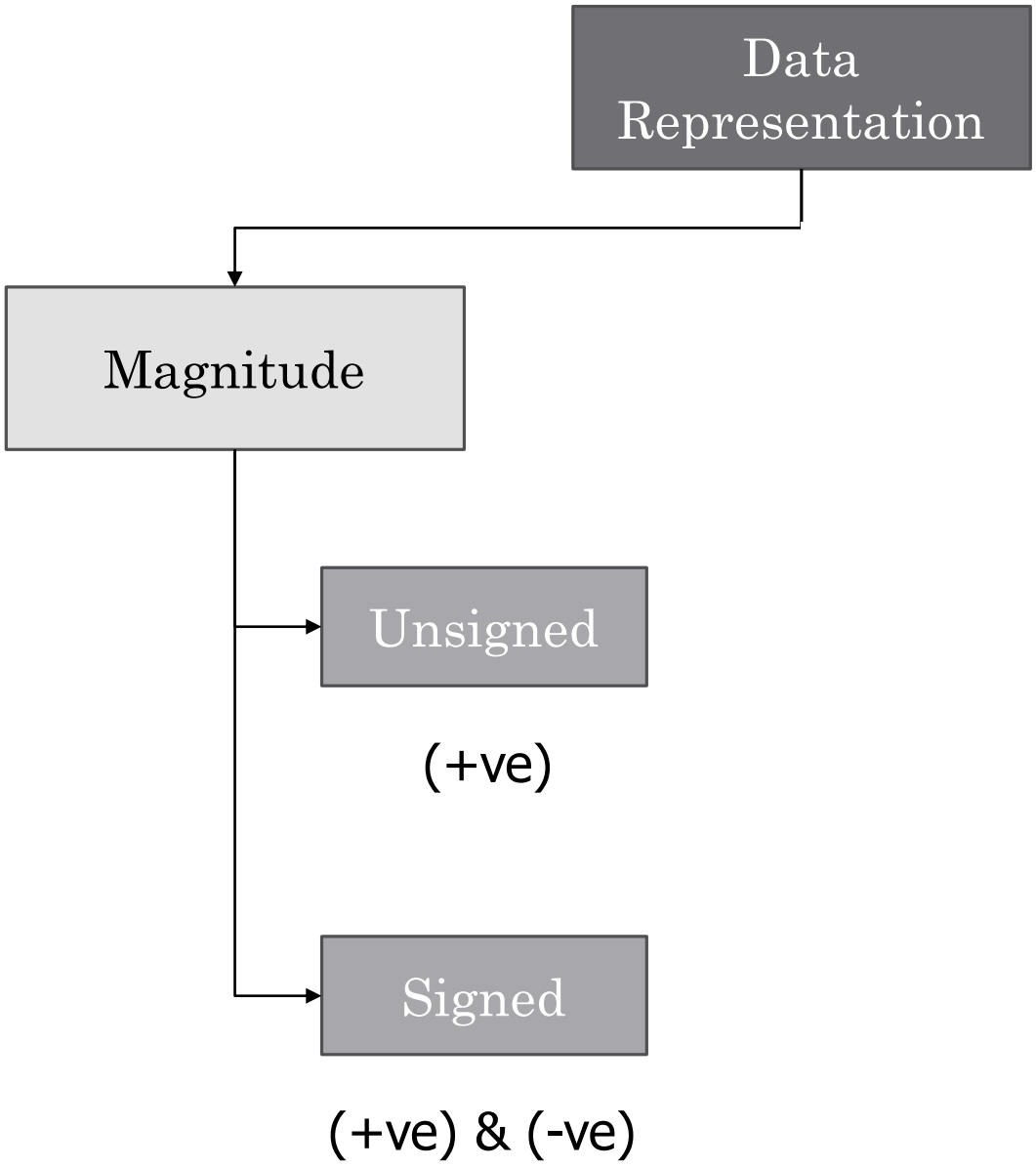
00101001 = 41

10000000 = 128

11111111 = 255

Magnitude

complement



In all representation, (+) number is represented in the same way

Unsigned magnitude

- Represents **positive integers**

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Pattern	1	0	0	1	1	1	0	1
Weight	128	64	32	16	8	4	2	1

10011101 = unsigned representation of 157

Unsigned magnitude

- Only accept positive numbers
- In computer programming, the unsigned integers property open for application in most numeric data types:
  - Short
  - Long
  - Int
  - Char

Advantages and disadvantages of unsigned magnitude

Advantages

- One representation of zero
- Simple addition

Disadvantages

- Negative numbers cannot be represented
- The need of different notation to represent negative numbers

Is a representation of negative numbers possible?

Unfortunately,

- You **cannot** just stick a negative sign in front of a binary number (it does not work like that)
- There are three methods used to represent negative numbers:

Signed magnitude notation

1s complement notation

2s complement notation

Signed-magnitude

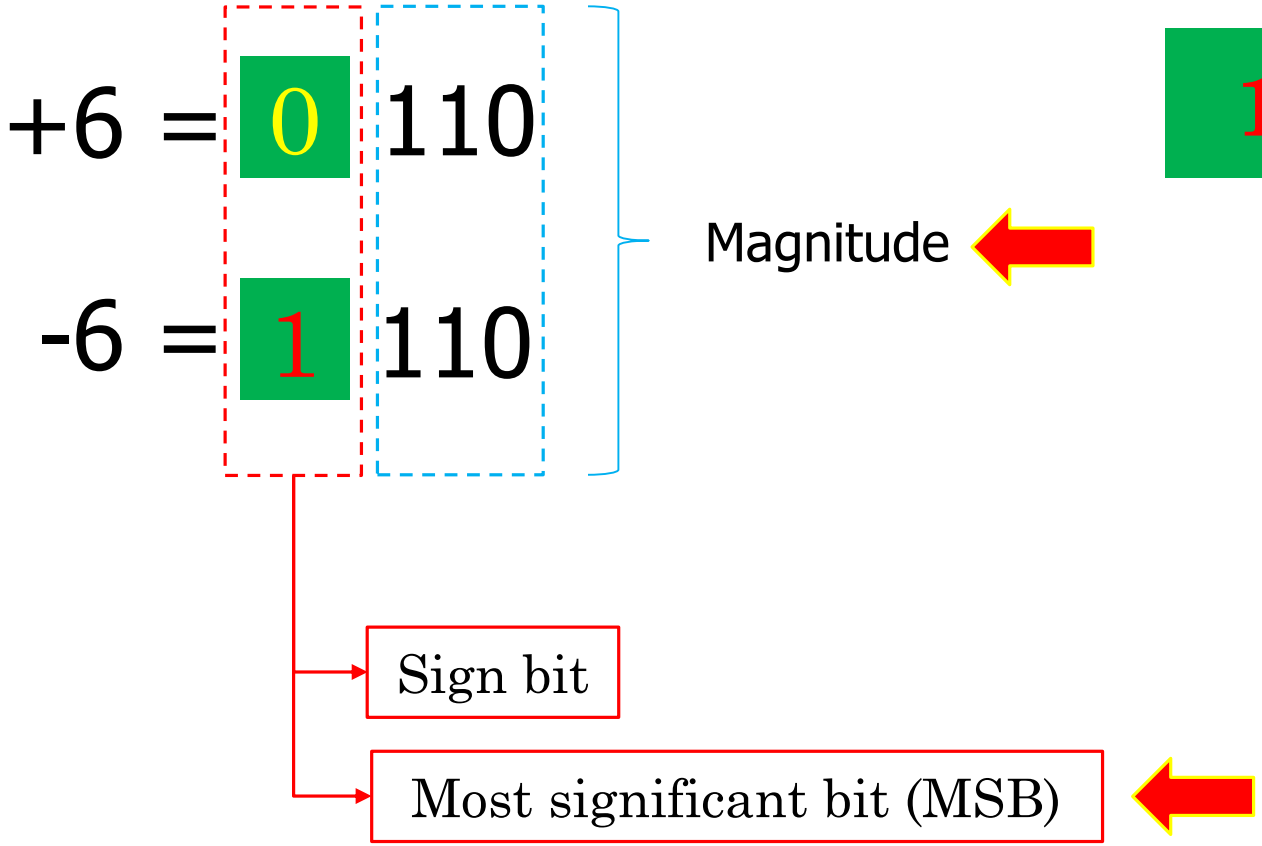
- Human use the signed-magnitude system.
- We add + or – to the front of a number to indicate its sign
- We can do this in binary too! By adding a sign bit in front of our numbers.

Signed-Magnitude

- Use **one-bit** to represent the sign of the integer
- Left most bit (Most Significant Bit, MSB) is **sign bit**

0 means **positive**	**1** means **negative**
+18 = **0**0010010	-18 = **1**0010010

Signed-Magnitude



Signed-Magnitude

0 = (+)

1 = (-)

+13 = 0 1101

-13 = 1 1101

Signed-magnitude

- Suppose 10011101 is a signed magnitude representation.
- The sign bit is 1, then the number represented is negative

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Pattern	1	0	0	1	1	1	0	1
Weight				16	8	4		1

- The magnitude is 0011101 with a value of **$2^4 + 2^3\ 2^2 + 2^0 = 29$**
- Then, the number represented by **10011101 is -29.**

Signed-Magnitude Range

- The range of signed magnitude representation depends on the number of bits used for both the sign bit and the magnitude part
- Sign Bit part: 1 bit (1 for negative, 0 for positive)
- Magnitude Part: (n-1) bits (assuming 1 bit is used for the sign)
- The range of values that can be represented in signed magnitude representation with n bits is typically:

Range of signed magnitude = $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$

n = number of bit

Signed-Magnitude Range

n = number of bit

$$-(2^{n-1}-1) \text{ to } +(2^{n-1}-1)$$

Example:

n = 4

$$-(2^{4-1}-1) \text{ to } +(2^{4-1}-1)$$

$$-(2^{3}-1) \text{ to } +(2^{3}-1)$$

$$-(2^{3}-1) \text{ to } +(2^{3}-1)$$

$$-(8-1) \text{ to } +(8-1)$$

Answer:

The signed magnitude for 4 bit are between:

- (7) to + (7)

Drawbacks of sign-magnitude

- Addition and subtraction require a consideration of both the signs of the numbers and their relative magnitudes to carry out the required operation.
- They are two representations of 0

$$00000000 = +0_{10}$$
$$10000000 = -0_{10}$$

- To test if a number is 0 or not, the CPU will need to see whether it is 00000000 or 10000000.
- Therefore, having 2 representation of 0 is inconvenient.

Quick Activity #1

a. 37_{10} has 00100101 is signed magnitude notation. Find the signed magnitude of -37_{10}.

b. Using the signed magnitude notation find the 8-bit binary representation of the decimal value 24_{10} and -24_{10}.

c. Find the signed magnitude of –63 using 8-bit binary sequence?

Quick Activity #1

① 37_{10} = 00100101 (+ve)

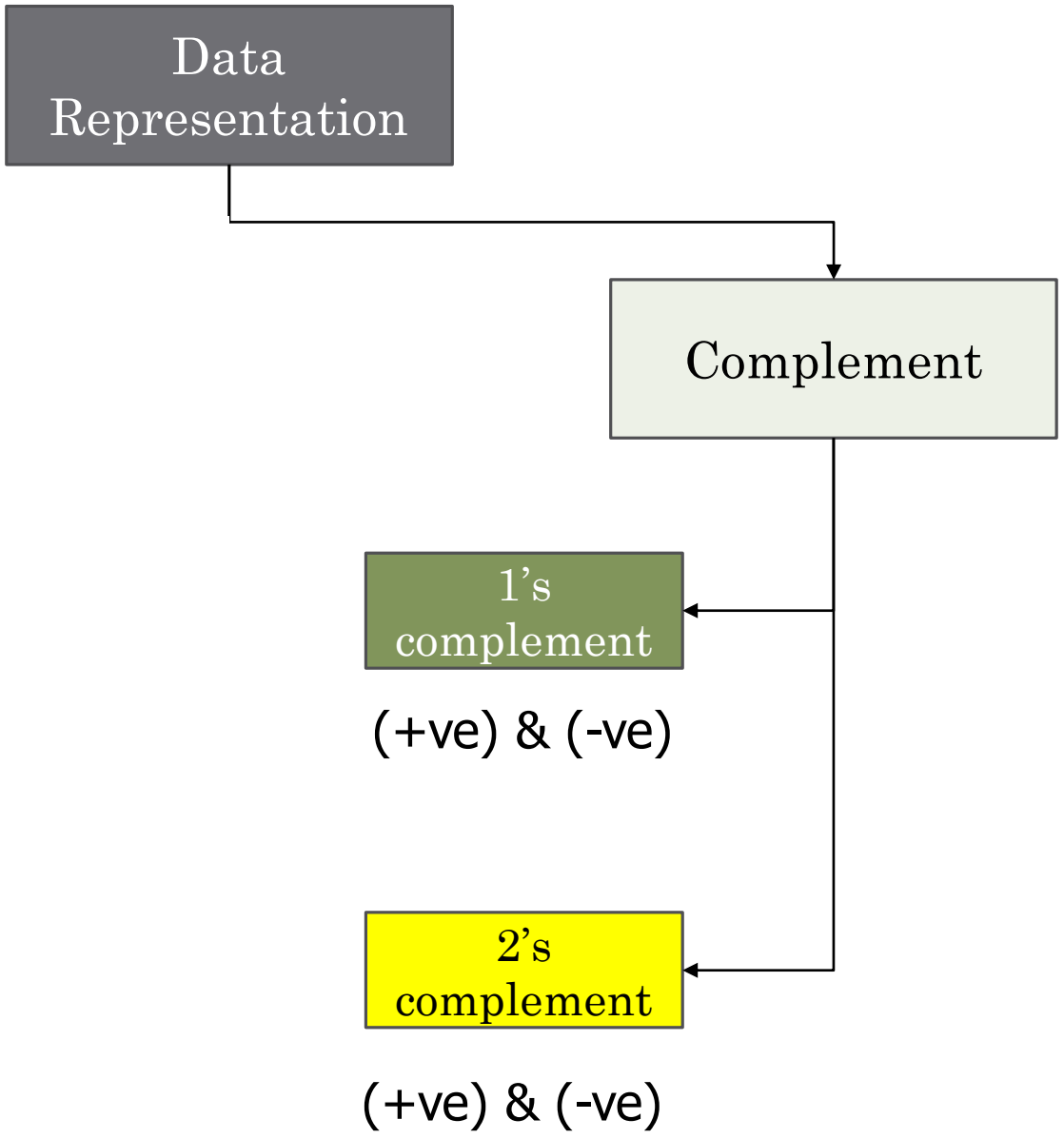
-37_{10} = 1010010

② 8-bit representation of 24_{10} & -24_{10}

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
0	0	0	1	1	0	0	0	$= 24_{10}$
1	0	0	1	1	0	0	0	$= -24_{10}$

③ 8-bit of -63_{10} = 10111111_2

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	1	1	1	1	1	1



In all representation, (+) number is represented in the same way

1s complement

- In a different representation, 1s complement, we negate numbers by complementing each bit of the number.
- We keep the sign bits; 0 for positive and 1 for negative
- The sign bit is complemented along with the rest of the bits

1's complement

- Negative numbers are represented by taking the bitwise complement (**inverting all the bits**) of the positive binary representation of the number

$$(1010)_2 = (0101)_2$$

2’s complement

- 2's complement is a binary representation method used in computers to represent signed integers (both positive and negative numbers).
- It is the most common method for representing signed integers
- 2’s complement requires 2 rules:

Rule 1 — Find 1’s complement for the integer

Rule 2 — Add 1 to the 1’s complement

Step 1: Find 1's complement for the integer
Step 2: Add 1 to the 1's complement

2's complement

Example:

$(10111010)_2$

Step 1

$$01000101$$

Step 2

$$\begin{array}{r} 01000101^{\,1} \\ + \qquad\quad 1 \\ \hline 01000110 \end{array}$$

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 carry forward 1

2's complement (Alt)

+3 = 00000011
+2 = 00000010
+1 = 00000001
+0 = 00000000
-1 = 11111111
-2 = 11111110
-3 = 11111101

1. Most significant bit (MSB) is **sign bit** (Left)
2. **One** representation of **zero** = 1's complement
3. **Add 1 to LSB** (Right)

2's complement

1. +3 = 00000011
2. 1's complement
   11111100
3. Add 1 to LSB
   11111101

+3 = 00000011
+2 = 00000010
+1 = 00000001
+0 = 00000000
-1 = 11111111
-2 = 11111110
-3 = 11111101

2's complement (Alt)

+3 = 00000011
+2 = 00000010
+1 = 00000001
+0 = 00000000
-1 = 11111111
-2 = 11111110
-3 = 11111101

1. Identify the LSB and copy all zeros until the first 1
2. Copy the first 1
3. **Complement all the remaining bits (after 1)**

2’s complement

1

+3 = 00000011

2

11111101

+3 = 00000011
+2 = 00000010
+1 = 00000001
+0 = 00000000
-1 = 11111111
-2 = 11111110
-3 = 11111101

Quick Activity #2

- 01001_2
- 01011_2
- 00111_2
- 00001_2

Characteristics of 2's Complement Representation

Range	-2^{n-1} through $2^{n-1} - 1$
Number of Representations of Zero	One
Negation	Take the Boolean complement of each bit of the corresponding positive number, then add 1 to the resulting bit pattern viewed as an unsigned integer.
Expansion of Bit Length	Add additional bit positions to the left and fill in with the value of the original sign bit.
Overflow Rule	If two numbers with the same sign (both positive or both negative) are added, then overflow occurs if and only if the result has the opposite sign.
Subtraction Rule	To subtract *B* from *A*, take the twos complement of *B* and add it to *A*.

n = number of bit

$$-(2^{n-1}) \text{ to } +(2^{n-1} - 1)$$

Advantages of 2's complement

- It eliminates the need for two representations of zero, resulting in a unique representation.
- Arithmetic operations like addition, subtraction, and multiplication are simplified, and there are no issues with end-around carries.
- The representation is symmetric, meaning the range of positive and negative values is evenly distributed around zero.
- It's widely supported and used in modern computer systems due to its efficiency in arithmetic operations and other benefits.

Advantages of 2's complement

Subtraction can be easily performed

Multiplication is just a repeated addition

Division is just a repeated subtraction

2s complement is widely used in ALU

Converting Between 2's Complement Binary To Decimal

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Weight	128	64	32	16	8	4	2	1
Bit Pattern								

An eight-position 2's complement value box

Converting Between 2's Complement Binary To Decimal

Example:
Convert binary 10000011 to decimal

128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

-128 +2 +1

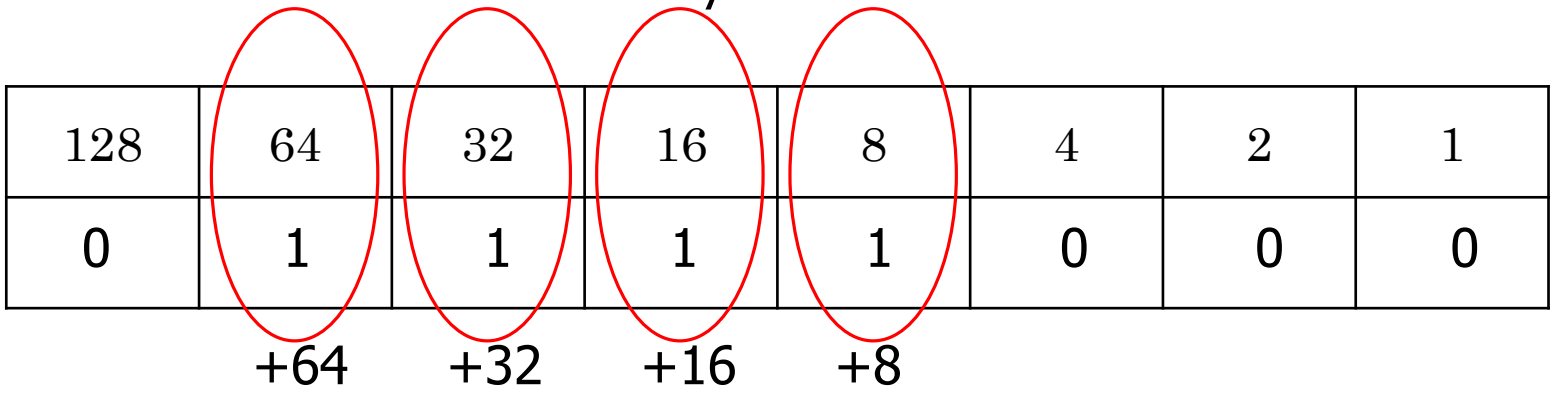
10000011 = -128 + 2 + 1

10000011 = -128 + 3

10000011 =-125

Converting Between 2's Complement Decimal To Binary

Example:
Convert decimal 120 to binary



128	64	32	16	8	4	2	1
0	1	1	1	1	0	0	0

+64 +32 +16 +8

64 + 32 + 16 + 8= 01111000

120 = 01111000

Converting Between 2's Complement Decimal To Binary

Example:
Convert decimal -120 to binary

-128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

-128 +8

-128 + 8 = 10001000

-120 = 10001000

In-class activity

Convert the following:

10100101 66

01101101 -80

128	64	32	16	8	4	2	1

Negation

- In signed-magnitude representation, the rule for forming the negation of an integer is simple: invert the sign bit.
- In twos complement notation, the negation of an integer can be formed with the following rules:

1. Find 1's complement to the integer
2. Add 1 to 1's complement integer

$$
\begin{aligned}
+18 &= 00010010 \quad \text{(twos complement)} \\
\text{bitwise complement} &= 11101101 \\
&\underline{+\qquad 1} \\
&11101110 = -18
\end{aligned}
$$

$$
\begin{aligned}
-18 &= 11101110 \quad \text{(twos complement)} \\
\text{bitwise complement} &= 00010001 \\
&\underline{+\qquad 1} \\
&00010010 = +18
\end{aligned}
$$

$$
\begin{aligned}
0 &= 00000000 \quad \text{(twos complement)} \\
\text{bitwise complement} &= 11111111 \\
&\underline{+\qquad 1} \\
&100000000 = 0
\end{aligned}
$$

$$
\begin{aligned}
+128 &= 10000000 \quad \text{(twos complement)} \\
\text{bitwise complement} &= 01111111 \\
&\underline{+\qquad 1} \\
&10000000 = -128
\end{aligned}
$$

Quick Activity #3

Represent these numbers in 8-bit 2's complement

a. + 44
b. + 89
c. + 106
d. - 18
e. - 74
f. - 86
g. - 99
h. - 110
i. - 120
j. - 128

2's complement operations

Addition

Subtraction

Multiplication

Division

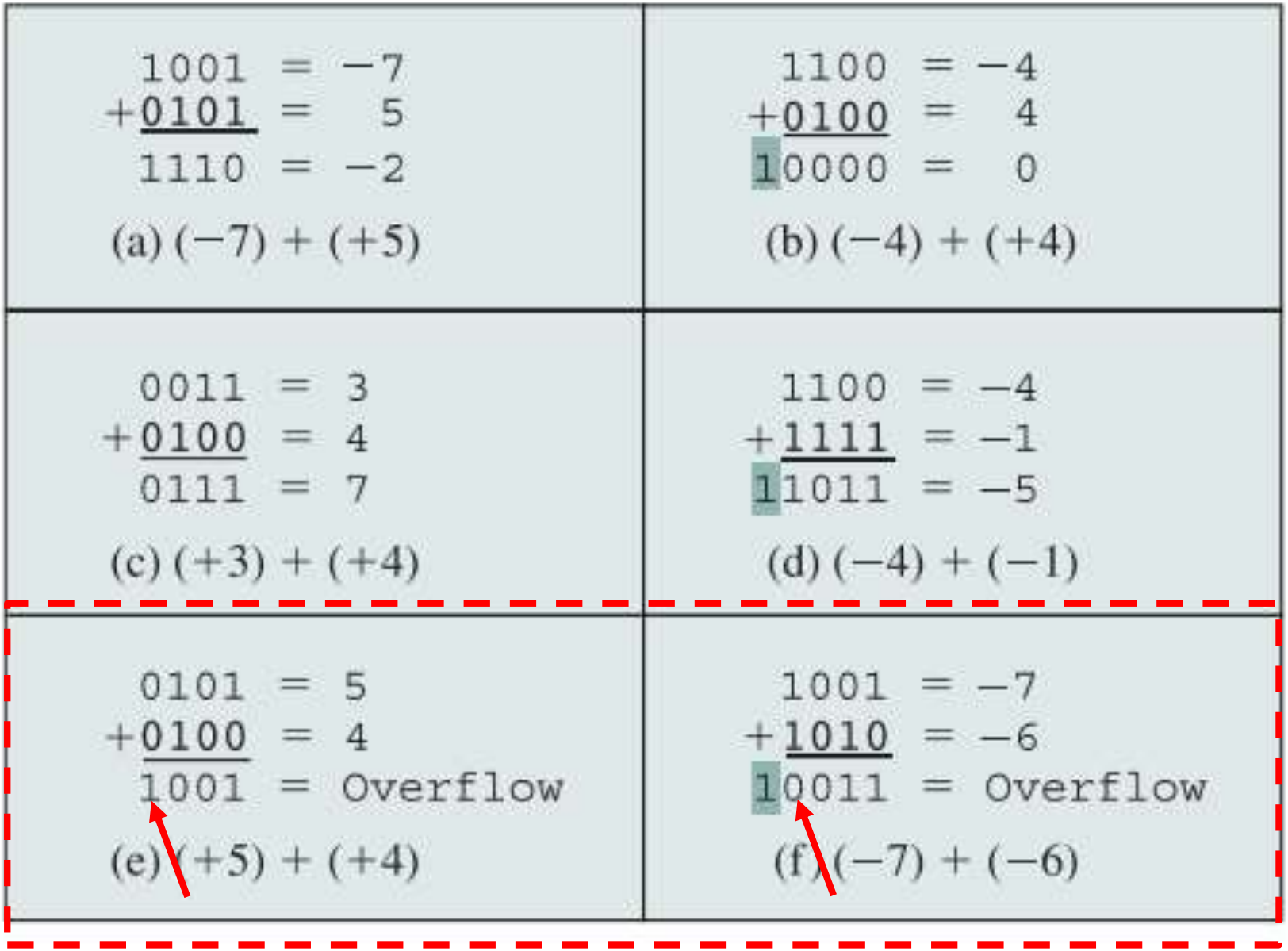
Addition in 2's in complement

1001 = −7 +0101 = 5 1110 = −2 (a) (−7) + (+5)	1100 = −4 +0100 = 4 10000 = 0 (b) (−4) + (+4)
0011 = 3 +0100 = 4 0111 = 7 (c) (+3) + (+4)	1100 = −4 +1111 = −1 11011 = −5 (d) (−4) + (−1)
0101 = 5 +0100 = 4 1001 = Overflow (e) (+5) + (+4)	1001 = −7 +1010 = −6 10011 = Overflow (f) (−7) + (−6)

Addition and Subtraction

- Addition proceeds as if the two numbers were unsigned integers
- If the **result** of the operation is **positive**, we get a **positive** number **in 2's complement** form, which is the same as in unsigned integer form
- If the **result** of the operation is **negative**, we get a **negative** number in **2's complement** form
- In some instances, there is a **carry bit** beyond the end of the word (indicated by shading), which is **ignored**
- On any addition, the result may be larger than can be held in the word size being used = **OVERFLOW**
- When overflow occurs, the ALU must signal this fact so that no attempt is made to use the result

0 +
1-


1001 = −7
+0101 = 5
1110 = −2
(a) (−7) + (+5)
1100 = −4
+0100 = 4
10000 = 0
(b) (−4) + (+4)
0011 = 3
+0100 = 4
0111 = 7
(c) (+3) + (+4)
1100 = −4
+1111 = −1
11011 = −5
(d) (−4) + (−1)
0101 = 5
+0100 = 4
1001 = Overflow
(e) (+5) + (+4)
1001 = −7
+1010 = −6
10011 = Overflow
(f) (−7) + (−6)

Rules of Binary Arithmetic

Binary Addition

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 0 Sum of 0 with a carry of 1

Binary Subtraction

- 0 – 0 = 0
- 1 – 0 = 1
- 1 – 1 = 0
- 0 = 1 = 1 (Borrow 1)

Binary Multiplication

- 0 × 0 = 0
- 0 × 1 = 0
- 1 × 0 = 0
- 1 × 1 = 1

Addition (Example 1)

Add two 4-bits 2's complement of 4 and 3

Step 1 – Convert 4 and 3 to binary

	128	64	32	16	8	4	2	1
4					0	1	0	0
3					0	0	1	1

Step 2 – Perform binary addition

4					0	1	0	0
3					0	0	1	1
					0	1	1	1

Answer: 4 + 3 = 7 = 0111

Addition (Example 2)

Add two 5-bit 2's complement of 13 and -6

Step 1 – Convert 13 and -6 to binary

	128	64	32	16	8	4	2	1
13				0	1	1	0	1

	128	64	32	16	8	4	2	1
+6				0	0	1	1	0
-6				1	1	0	1	0

In 2s complement

Step 2 – Perform binary addition

13			0	1	1	0	1
-6			1	1	0	1	0
		1	0	0	1	1	1

Answer: 13 + (-6) = 7 = 00111

Addition (Example 3)

Add two 5-bit 2's complement of -15 and 9

Step 1 – Convert -15 and 9 to binary

	128	64	32	16	8	4	2	1
+15				0	1	1	1	1
-15				1	0	0	0	1

In 2s complement

	128	64	32	16	8	4	2	1
9				0	1	0	0	1

Step 2 – Perform binary addition

15			1	0	0	0	1
-6			0	1	0	0	1
			1	1	0	1	0

Answer: -15 + 9= -6 = 11010

Addition (Example 4)

Add two 8-bit 2's complement of -39 and 92

Step 1 – Convert -39 and 92 to binary

	128	64	32	16	8	4	2	1
+39	0	0	1	0	0	1	1	1
-39	1	1	0	1	1	0	0	1

In 2s complement

	128	64	32	16	8	4	2	1
92	0	1	0	1	1	1	0	0

Step 2 – Perform binary addition

-39	1	1	0	1	1	0	0	1
92	0	1	0	1	1	1	0	0
1	0	0	1	1	0	1	0	1

Answer: -39 + 92= 53= 00110101

Addition (Example 5)

Add two 5-bit 2's complement of -8 and -4

Step 1 – Convert -8 and -4 to binary

128	64	32	16	8	4	2	1
+8			0	1	0	0	0
-8			1	1	0	0	0
+4			0	0	1	0	0
-4			1	1	1	0	0

Step 2 – Perform binary addition

-8			1	1	0	0	0
-4			1	1	1	0	0
		1	1	0	1	0	0

Answer: -8 + (-4)= -12 = 10100

Addition (Example 4)

Overflow example

Add two 8-bit 2's complement of 104 + 45

Step 1 – Convert 104 and 45 to binary

	128	64	32	16	8	4	2	1
104	0	1	1	0	1	0	0	0
45	0	0	1	0	1	1	0	1

Step 2 – Perform binary addition

104	0	1	1	0	1	0	0	0
45	0	0	1	0	1	1	0	1
	1	0	0	1	0	1	0	1

Answer: 104 + 45 = 149 = 1[illegible]

How do we handle overflow?

- Since we added two positive numbers and got a negative result, overflow has indeed occurred. Therefore, this answer is **not correct** for 8-bit representation.
- So, the correct answer cannot be represented within an 8-bit signed integer and would **require a larger bit width** (like 9 bits or more) to hold the value accurately.

Quick Activity #4

Assume numbers are represented in 8-bit twos-complement representation. Show the calculation of the following:

1. -19 + (-7)
2. 44 + 45
3. -75 + 59
4. 127 + 1
5. -1+1
6. -103 + (-69)

Decimal Representation	Sign-Magnitude Representation	Twos Complement Representation	Biased Representation
+8	—	—	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
−0	1000	—	—
−1	1001	1111	0110
−2	1010	1110	0101
−3	1011	1101	0100
−4	1100	1100	0011
−5	1101	1011	0010
−6	1110	1010	0001
−7	1111	1001	0000
−8	—	1000	—

2's complement operations

Addition

Subtraction

Multiplication

Division

0 +
1-

Subtraction in 2's complement

To subtract one number (subtrahend) [S] from another (minuend) [M], take the 2s complement (negation) of the subtrahend and add them to the minuend

[M] + 2s [S]

Step 1: Find 2s complement of subtrahend [S]

Step 2: Perform the addition

Step 3: If the carry bit is generated and the result is positive, and in its true form

If carry bit is not produced, then the result is negative and in its 2s complement form

Subtraction (Example 1)

Overflow example

Subtract two 4-bit 2's complement of 2 and -7

M = 2
S = -7

Step 1 – Convert 2 and -7 to binary

M	128	64	32	16	8	4	2	1
2					0	0	1	0

S	128	64	32	16	8	4	2	1
7					0	1	1	1
-7					1	0	0	1

Step 2 – Perform binary addition

+2					0	0	1	0
+7					0	1	1	1
					1	0	0	1

Answer: 2 –(-7) = 2 + 7 = 9

Subtraction (Example 2)

Subtract two 4-bit 2's complement of 5 and -2

M = 5
S = -2

Step 1 – Convert 5 and -2 to binary

M	128	64	32	16	8	4	2	1
5					0	1	0	1

S	128	64	32	16	8	4	2	1
2					0	0	1	0

Step 2 – Perform binary addition

5					0	1	0	1
2					0	0	1	0
					0	1	1	1

Answer: 5 – (-2) = 5 + 2 = 7 = 0111

Answer: 5 –(-3) = 5 +3 = 8

Overflow example

Subtract two 4-bit 2's complement of 5 and -3

M =5
S =-3

Step 1 – Convert -5 and -2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1

S	128	64	32	16	8	4	2	1
+3					0	0	1	1

Step 2 – Perform binary addition

+5					0	1	0	1
+3					0	0	1	1
				1	0	0	0	0

Subtraction (Example 4)

Subtract two 4-bit 2's complement of 5 and 2

M =5
S =2

Step 1 – Convert 5 and 2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1
+2					0	0	1	0
-2					1	1	1	0

Step 2 – Perform binary addition

5					0	1	0	1
-2					1	1	1	0
				1	0	0	1	1

Answer: 5 – (+2) = 5 – 2 = 3 = 0011

Subtraction (Example 5)

Subtract two 4-bit 2's complement of 7and 7

M =7
S =7

Step 1 – Convert 7 to binary

M	128	64	32	16	8	4	2	1
7					0	1	1	1
-7					1	0	0	1

Step 2 – Perform binary addition

7					0	1	1	1
-7					1	0	0	1
				1	0	0	0	0

Answer: 7– 7= 0 = 10000

Subtraction (Example 6)

Overflow example

Subtract two 4-bit 2's complement of 7and -7

M =7
S =-7

Step 1 – Convert 7 to binary

M	128	64	32	16	8	4	2	1
7					0	1	1	1

Step 2 – Perform binary addition

7					0	1	1	1
7					0	1	1	1
					1	1	1	0

Answer: 7–(-7) = 14 = 1110

Answer: 5 – (-6) = 5 + 6 = 11

Overflow example

Subtract two 4-bit 2's complement of 5 and -6

M =5
S =-6

Step 1 – Convert -5 and -2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1

S	128	64	32	16	8	4	2	1
+6					0	1	1	0

Step 2 – Perform binary addition

+5					0	1	0	1
+6					0	1	1	0
					1	0	1	1

Quick Activity #5

Assume numbers are represented in 8-bit twos-complement representation. Show the calculation of the following:

i. 5 – 3

ii. 7 – 4

iii. −6−2

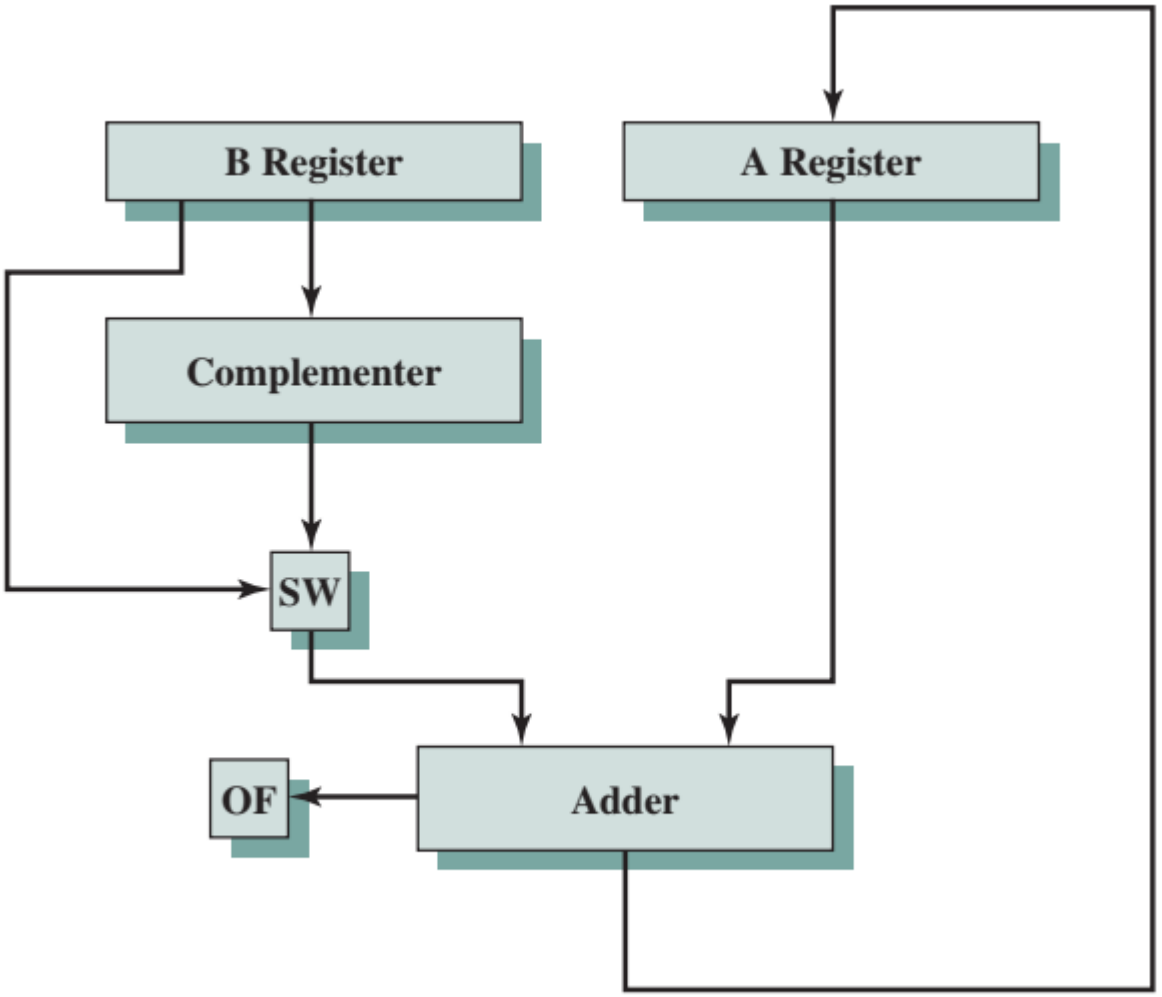
iv. 12−(−5)

v. −7−(−8)

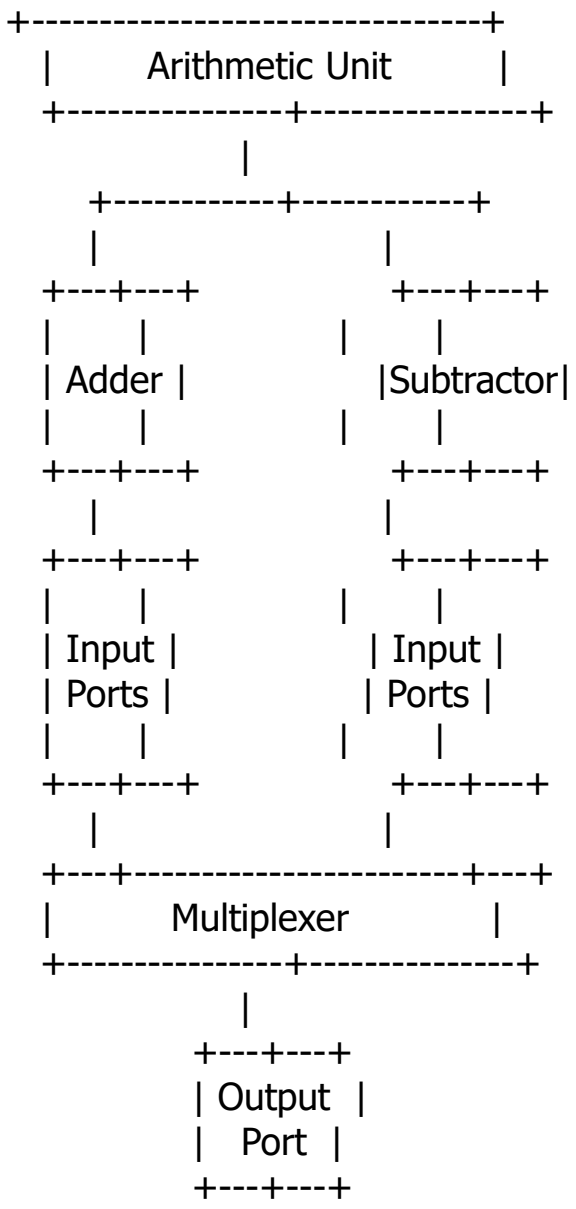
vi. 45−18

vii. −50−25

Hardware for Addition and Subtraction



OF = Overflow bit
SW = Switch (select addition or subtraction)



1. The control signals for the multiplexer determine whether the arithmetic unit performs addition or subtraction.
2. The output of the selected operation is then passed to the output port.
3. This basic hardware setup can be expanded and optimized for more complex arithmetic operations and can be part of a larger CPU or digital processing system.